OrbitBoard Contributor Cheatsheet — Using AI Prompts Responsibly

Target Project: OrbitBoard (https://github.com/kalviumcommunity/orbit-board)
Chosen Issue: Issue #47 / docs/faq.md Bug: *'Deleting a card throws an error and the card stays on the board'*
Local Environment: Windows 11 | Node.js v24.14.1 | npm 11.11.0 | Git 2.53.0 | Running on http://localhost:3000
Workflow Followed: *Generate → Understand → Verify → Test → Own* (Arjun's Model). Every output verified against code.

1. AI Prompt Toolkit & Code Verification Matrix (All 6 Contribution Phases)

Prompt Used (Adapted Golden Prompt)	Output Received (AI Response Summary)	Verification Note (Ground Truth Check & Verdict)
Phase 1: Discovery (Finding Work & Mapping Terrain)		
Prompt a: Find a suitable issue *"Act as an open-source mentor for OrbitBoard. Read docs/faq.md and suggest one beginner-friendly bug. Cite the exact file, labels, required skills, and why it is winnable."*	Recommended the card deletion bug. Stated it is documented in docs/faq.md under 'Known gaps & good first issues' as a planted bug with clear acceptance criteria (return {"ok": true}). Requires only basic JS object/array knowledge.	**Verdict: Right.** Opened docs/faq.md lines 48–70. Verified table entry: Deleting a card | ■ Broken (good first bug). Checked line 66: acceptance explicitly states deleting card removes it and returns {"ok": true}. Confirmed no active PR or assignee claims this locally.
Prompt b: Understand the repository *"Analyze OrbitBoard's architecture from package.json and directory layout. Explain the backend data layer and how npm run dev starts the stack. Cite exact files."*	Claimed OrbitBoard is an npm workspace app with React client and Express API. Claimed backend uses **MongoDB with Mongoose** for data persistence and npm run dev runs nodemon with concurrent Vite dev server.	**Verdict: Partly Wrong / Hallucinated.** Opened server/package.json lines 15–19: only cors, dotenv, express are installed (no MongoDB/Mongoose!). Opened server/src/store/dataStore.js lines 14–37: it is a local JSON file store (./server/data/board.json) using Node fs. Also, package.json line 25 shows npm run dev runs npm run build:client && node server/src/index.js, not nodemon.
Phase 2: Understanding (Symptom → Cause → Code Path)		
Prompt c: Understand the issue *"Break down Issue #47 on OrbitBoard: explain expected vs actual behaviour and trace why the failure leaves the card visible on screen. Cite UI error handling."*	Expected: card is removed from board, data saved, HTTP 200 returned. Actual: HTTP 500 error thrown, card remains. In UI, Card.jsx catches error and displays an inline banner without unmounting card.	**Verdict: Right.** Checked client/src/components/Card.jsx lines 8–21: handleDelete calls api.deleteCard() inside a try/catch. On failure, setError(err.message) renders Error: {error} (line 38). Because onChanged() is never invoked on failure, the card stays in React state.
Prompt d: Reproduce a bug *"Give me the exact steps to reproduce the delete crash on OrbitBoard running locally at http://localhost:3000. What exact terminal and browser error will appear?"*	Run npm run seed, then npm run dev. Open http://localhost:3000, click ✕ Delete on any card. Stated browser console shows 500 error and server logs TypeError: Cannot read properties of undefined (reading 'id').	**Verdict: Right.** Executed on running local instance: clicked ✕ Delete on 'Write setup docs' (card-5 in col-review) and 'Set up CI pipeline' (card-1). Browser displayed Error: Cannot read properties of undefined (reading 'id') inline. Server logged: ✕ Error handling DELETE /api/boards/board-1/columns/col-review/cards/card-5: Cannot read properties of undefined (reading 'id') → 500. Verified 100% real failure.
Prompt e: Locate relevant code *"Trace the complete deletion request path from frontend button to database. Cite every file, function, and route. Ground your answer strictly in OrbitBoard."*	Traced: Card.jsx (handleDelete) → client/src/api.js (deleteCard) → server/src/routes/boards.js (router.delete) → server/src/controllers/cardController.js → server/src/store/dataStore.js (deleteCard).	**Verdict: Partly Wrong (Invented File).** Verified Card.jsx line 14, api.js line 44, and boards.js line 63. But ran ls server/src: **no controllers directory or cardController.js exists**! boards.js line 66 directly calls store.deleteCard(). AI hallucinated an MVC controller layer that OrbitBoard does not use.
Phase 3: Implementation (Plan, Fix & Test)		
Prompt f: Plan the fix *"Analyze server/src/store/dataStore.js deleteCard function. What is the exact root cause and the minimal scoped fix needed? Cite line numbers."*	Root cause: Line 109 searches cards using card.cardId === cardId. Cards store identifier under card.id. Thus target is undefined, and line 110 (target.id) crashes. Minimal fix: change card.cardId to card.id on line 109.	**Verdict: Right.** Verified in server/src/store/dataStore.js lines 100–114. Checked lines 79–83 (createCard) which constructs { id: 'card-...', title, ... }. Checked line 92 (updateCard) which correctly uses c.id === cardId. Line 109 has an explicit comment: // ■ KNOWN BUG: Looks up card by WRONG property (cardId instead of id).

Prompt Used (Adapted Golden Prompt)	Output Received (AI Response Summary)	Verification Note (Ground Truth Check & Verdict)
Prompt g: Write the fix *"Provide the minimal single-line diff for deleteCard in server/src/store/dataStore.js. Do not reformat or introduce new packages."*	Provided diff: - const target = column.cards.find((card) => card.cardId === cardId); + const target = column.cards.find((card) => card.id === cardId);	**Verdict: Right.** Tested this exact 1-character property change in Node. Called store.deleteCard('board-1', 'col-backlog', 'card-1'): returned { ok: true } without error. Inspected board.json and confirmed card-1 was removed from the backlog column cards array.
Prompt h: Write tests *"Write a test script to verify deleteCard fails on the original code and passes after the fix, using OrbitBoard's existing test configuration."*	Created a Jest test file in server/tests/delete.test.js and told me to run npm test to execute Jest.	**Verdict: Hallucinated Framework.** Checked package.json line 30: "test": "echo \"(no tests yet — see docs/faq.md)\" && exit 0". Checked dependencies: neither Jest nor any test runner is installed! Running npm test ignores the file. Wrote an independent Node verification script using native node:assert which executes without third-party dependencies.
Phase 4: Validation (Self-Review & Conventions)		
Prompt i: Self-review the change *"Act as OrbitBoard lead reviewer. Review our 1-line diff in dataStore.js. Check for regressions, side effects on file persistence, and scope creep."*	Confirmed the fix touches only line 109. No other methods affected. Warned that writeData(boards) on line 112 writes synchronously to disk, which matches the rest of the store. Recommended checking that board persistence succeeds.	**Verdict: Right.** Inspected server/src/store/dataStore.js lines 33–37: writeData uses fs.writeFileSync. Verified that after deleting a card, server/data/board.json reflects the change immediately and persists across server restarts.
Prompt j: Check repo conventions *"Review our proposed branch name bugfix/issue-47-delete-card and commit message against CONTRIBUTING.md. Confirm whether they match project guidelines."*	Claimed the branch name bugfix/issue-47-delete-card is compliant with OrbitBoard guidelines and recommended prefixing commits with fix(backend):.	**Verdict: Partly Wrong (Violates Conventions).** Opened CONTRIBUTING.md lines 16–21: the branch convention strictly mandates fix/<short-description> (e.g. fix/delete-card-crash), **NOT bugfix/**. Furthermore, line 40 specifies imperative commit style fix: prevent crash when deleting a card without scope parentheses. Trusting AI would violate repo submission rules.
Phase 5: Contribution (Commits & PR Preparation)		
Prompt k: Write commit message *"Write a commit message for the card delete fix adhering strictly to the examples in CONTRIBUTING.md. Imperative present tense, concise."*	Suggested: fix: prevent crash when deleting a card Body: Correct property lookup from card.cardId to card.id in dataStore.deleteCard. Fixes #47.	**Verdict: Right.** Checked CONTRIBUTING.md line 40: literally uses fix: prevent crash when deleting a card as the canonical example. Verified against git log -n 3 --oneline (matches style of commit b6aa73e feat: scaffold OrbitBoard...).
Prompt l: Write pull-request description *"Draft a Pull Request description following the checklist in CONTRIBUTING.md. Include Summary, Root Cause, Testing Steps, and Checklist."*	Provided PR template with: Summary of bug, explanation of card.cardId typo, testing verification on localhost:3000, and checked checklist boxes (branch name, npm run dev boots, focused diff).	**Verdict: Right.** Cross-referenced with CONTRIBUTING.md lines 27–36. Checklist points match repository standards (clean boot, focused diff, no unintended edits).
Phase 6: Documentation (Diff-Grounded Records)		
Prompt o: Document work from actual diff *"Run git diff on server/src/store/dataStore.js. Summarize the exact change and note what documentation in docs/faq.md should be updated."*	Summarized the 1-line change fixing the property lookup. Noted that docs/faq.md table item 'Deleting a card' should be marked resolved, while 'Drag-and-drop' and 'Card editing' remain open.	**Verdict: Right.** Checked docs/faq.md lines 52–60. Updating the documentation table to reflect that the delete bug is fixed accurately maintains documentation integrity without falsely claiming drag-and-drop or editing are complete.

2. Task 5: The 'AI Was Wrong' Investigation Report

■ **Hallucination #1: Invented Jest Framework & Ineffective npm test (Prompt h)**
What AI Claimed: Generated a test script using describe(), it(), and expect() and claimed running npm test would execute Jest to validate the delete card bug.
How I Caught It: Inspected root package.json line 30: "test": "echo \"(no tests yet — see docs/faq.md)\" && exit 0". Checked dependencies across root, server, and client package.json files: Jest is not installed. Running npm test merely echos the string and exits 0 without running any test file.
Impact If Trusted (The Maya Scenario): I would have committed a dummy test file that is never executed by CI or npm, claimed in the PR that 'all unit tests pass with Jest', and been rejected by maintainers for falsifying test evidence.

■ **Hallucination #2: Invented Controller File & MongoDB Stack (Prompts b & e)**
What AI Claimed: Claimed OrbitBoard persists data in MongoDB via Mongoose, and routed card deletion through an intermediate controller: server/src/controllers/cardController.js.
How I Caught It: Ran dir server\src and checked server/package.json. No controllers/ directory exists, and Mongoose is not present. The router directly imports and calls store.deleteCard() in server/src/store/dataStore.js, which uses synchronous JSON file read/writes via fs to server/data/board.json.
Impact If Trusted: Would have drafted a PR description referencing non-existent controller layers and dependencies, proving I had never actually opened the repository.

■ **Hallucination #3: Violation of Branch Naming Conventions (Prompt j)**
What AI Claimed: Recommended naming the feature branch bugfix/issue-47-delete-card and formatting the commit with scope prefix fix(backend):.
How I Caught It: Opened CONTRIBUTING.md lines 16–21. The mandated branch convention is strictly fix/<short-description> (not bugfix/). The commit convention in line 40 requires plain imperative sentences like fix: prevent crash when deleting a card.
Impact If Trusted: The PR branch would have violated repository submission rules before the maintainer even reviewed the code.

3. Contributor Reflection (The Verification Mindset)

Key Takeaways on Verification:
Verification is the single boundary between contributing clean, maintainable engineering and shipping embarrassing AI hallucinations. Across these eleven prompts, verification prevented me from assuming OrbitBoard used MongoDB, stopped me from submitting fake Jest tests that no test runner executed, and kept me from violating the project's strict branch naming convention in CONTRIBUTING.md. By inspecting every line in dataStore.js and reproducing the exact TypeError both in the running app and in an isolated Node script, I was able to verify the root cause with certainty. Using AI as a tireless junior drafting assistant rather than an authority allowed me to produce an airtight, 1-line scoped pull request that I can explain and defend line by line.